

BUS13-2

Designing Operations from Scratch

[Module page](#)

Version of 13 September 2026

About this textbook

This year, for the first time, every Bachelor course at Albert School has its own textbook, thanks to your teachers and AI.

- The textbook is a help for working on the course at home, after the class.
- The course as taught in class remains the reference, and prevails over the textbook wherever the two differ.
- This is a first version. Help us improve it by sharing your feedback with your teacher.

Contents

Preface	5
1. Operations as a function	6
1.1. The function that keeps the promise	6
1.2. How operations differs from its neighbours	6
1.3. The operating model	7
1.4. Company shapes: the structure that pre-decides the operation	7
2. Systems thinking as an operational lens	10
2.1. A warehouse that optimised itself into failure	10
2.2. Stocks and flows	10
2.3. Feedback, delay, and emergent behaviour	11
2.4. Suboptimization: the central warning	11
3. Constraints and the Theory of Constraints	13
3.1. One stage sets the pace	13
3.2. The Five Focusing Steps	13
3.3. Physical, policy, and organizational constraints	14
4. Throughput, utilization, and the cost of being busy	16
4.1. Why the 95%-busy team is the slow one	16
4.2. The three core measures	16
4.3. The utilization–delay curve	16
4.4. Slack as strategic capacity	18
5. Queues, flow, and Little’s Law	20
5.1. Counting a queue without a stopwatch	20
5.2. Little’s Law	20
5.3. One process, all three numbers	21
5.4. Visible and invisible queues	21
5.5. Batch size: the hidden delay multiplier	21
5.6. Rework: the hidden capacity tax	22
6. Designing a process from scratch	24
6.1. From a promise to a flow	24
6.2. Mapping the value delivery flow	24
6.3. Decision points and handoffs: where designs leak	24
6.4. Designing for clarity	25
7. Service design	26
7.1. The operation the customer actually feels	26
7.2. Touchpoints and the line of visibility	26
7.3. Failure points and recovery paths	26
8. Make vs. buy	28
8.1. Build it or rent it?	28
8.2. The true cost of each option	28
8.3. Coordination cost and the core/non-core line	29
9. Unit economics as operational constraints	31
9.1. The margin that bounds the design	31
9.2. The unit-economics toolkit	31
9.3. Fixed vs. variable: the shape of the cost structure	32
10. Operational MVP and failure-mode analysis	34

10.1. Shipping the smallest operation that proves the promise	34
10.2. The operational MVP	34
10.3. Early failure-mode analysis	35
10.4. Second-order effects: fixing one thing reveals the next	35
10.5. Presenting an operational design to a non-operations reader	36
Primary sources and further reading	37

Preface

Shown a business plan today, you can probably judge whether it sounds appealing. What you cannot yet do is judge whether the promise it makes can actually be **delivered** — repeatedly, at the volume and margin the plan assumes — or name where it will break first when it cannot. This course gives you that second kind of sight. By the end you will be able to:

- place operations among the business functions, and read how a venture's **company shape** — marketplace, SaaS, DTC, physical services — pre-decides the operation that must deliver it;
- see an operation as a system of stocks and flows, and spot where improving one part degrades the whole;
- find the single constraint that sets a system's pace, using the Theory of Constraints, and tell a physical constraint from a policy or organizational one;
- compute utilization and cycle time, and explain why variability, not busyness alone, is what makes systems slow;
- apply Little's Law to estimate lead time and work-in-progress, and show what batching and rework do to flow;
- design a process and its service experience end to end — decision points, handoffs, touchpoints, failure points, and recovery paths;
- weigh a make-vs-buy decision on total cost and dependency, not sticker price;
- turn given unit economics into hard bounds on what the operation may cost per unit;
- scope a minimum viable operation, name where and at what volume it breaks first, and justify the design in writing to a reader who does not speak operations.

Underneath all of it is a single reflex you do not have yet and will not lose: when someone describes a plan, you trace the flow, locate the constraint, and name the volume at which the design cracks. That reflex will outlast every formula in these pages.

1. Operations as a function

1.1. The function that keeps the promise

Every business makes a promise to a customer: a meal delivered hot, a software tool that is up whenever you open it, a pair of running shoes in your size on the shelf. **Marketing** makes the promise heard; **finance** funds and prices it; **strategy** chooses which promise to make, and to whom. But none of them **keep** it. Turning a customer's request into the thing they actually receive — repeatedly, at volume, and at a cost the business can bear — is the work of **operations**. It is the least glamorous function and the one that decides, more than any other, whether a venture survives contact with real demand.

Definition 1 (operations). **Operations** is the function that designs and runs the process by which a business turns inputs into the goods and services its customers receive. Where the other functions decide **what** to offer, **to whom**, and **at what price**, operations answers **how it is actually delivered**: the flow of work from a customer's request to a fulfilled order, and the people, steps, resources, and handoffs along the way.

A quick test of whether a question belongs to operations: **does answering it change how a unit of customer value physically moves through the business?** “Which segment do we target?” is strategy. “What do we charge?” is finance and marketing. “Who unpacks, checks, and ships the order, in what order, and what happens when an item is missing?” is operations. The other functions decide the promise; operations is where the promise meets reality.

1.2. How operations differs from its neighbours

The functions are easiest to separate by the question each one owns.

Function	The question it owns
Strategy	Which promise do we make, to whom, and why will we win?
Marketing	How do we make the promise heard and wanted ?
Finance	How do we fund and price it, and does it make money?
Operations	How is it actually delivered , every time, at volume and at cost?

Table 1: The business functions, separated by the question each owns. Operations is the only one accountable for whether the promise is physically kept — which is why a venture with brilliant strategy, marketing, and finance still dies if its operation cannot deliver at the volume the plan assumes.

These functions are not a hierarchy; they constrain one another. Finance's margin caps what operations may spend per order (Chapter 9); operations' real capacity caps what marketing may promise. A recurring failure of young ventures is a marketing department selling a delivery speed the operation was never built to hold — a promise made in one function and broken in another.

1.3. The operating model

Definition 2 (operating model). A venture's **operating model** is the concrete arrangement of people, process, and technology through which it delivers its product — who does what, in what order, with which systems and suppliers. The **org chart** shows **reporting** lines; the operating model shows **how value flows**. The two rarely match, and when a design goes wrong it is almost always the flow, not the chart, that reveals why.

Throughout this course you will design operating models, not org charts. The distinction matters immediately: the next chapter shows how looking at the reporting structure instead of the flow is the single most common way an operation optimises itself into failure.

1.4. Company shapes: the structure that pre-decides the operation

Before any design choice, one fact shapes the operation more than any other: the venture's **company shape** — the structural kind of business it is. Four shapes recur across almost every venture you will meet, and each **pre-decides** what the operation must be good at.

Framework 1 (the four company shapes).

- **Marketplace** — matches independent buyers and sellers it does not own (a resale app such as Vinted, a ride or delivery platform). The product is **liquidity**: enough of both sides that a match happens fast. Operations is mostly **matching, trust, and dispute handling** — the platform rarely touches the goods it sells.
- **SaaS** (software as a service) — sells ongoing access to software (a doctor-booking tool such as Doctolib, a CRM). The marginal cost of one more user is almost nothing; the operation is **uptime, onboarding, and support**, and the business lives or dies on reliability and retention, not on physical logistics.
- **DTC** (direct-to-consumer) — makes or sources a physical product and sells it straight to consumers, skipping the retailer (a challenger food brand such as Oatly). The operation is **supply, inventory, fulfilment, and returns**: real atoms, real warehouses, real breakage.
- **Physical services / retail** — delivers a service or sells goods through physical presence and staff (a sports-goods chain such as Decathlon, a clinic). The operation is **capacity, location, staffing, and the in-person experience**, all bounded by physical space and opening hours.

The word “delivery” means something different in each shape, and that difference **is** the operational challenge. Reading the shape tells you where the operation will strain first.

Shape	What “flows”	The binding operational question	Where it breaks first
Marketplace	matches	Can we make supply and demand meet fast and safely?	thin liquidity; fraud and disputes at scale
SaaS	sessions	Is it up, and do users get value before they churn?	downtime; onboarding that loses new users
DTC	physical units	Can we source, hold, ship, and take back goods at margin?	stockouts, breakage, and the cost of returns
Physical services	served customers	Do we have capacity in the right place at the right time?	queues at peak; idle, costly capacity off-peak

Table 2: How each company shape pre-decides the operation. The shape fixes what moves through the business, the question the operation must answer, and the failure that arrives first — long before any specific design choice is made.

Pitfall. The beginner treats “company shape” as a label for a pitch deck. It is not decoration; it is a **prediction of where the operation will strain**. A marketplace that pours its effort into warehousing (a DTC concern) has misread its own shape and starved the matching-and-trust problem that actually decides its survival. Name the shape first, and let it tell you which operational question is life-or-death — before you design anything.

Example — One venture, two shapes, two operations. A meal company can be built as **DTC** — cook and ship kits from a central kitchen — or as a **marketplace** connecting home cooks to nearby diners. Same food, same customer hunger, opposite operations. The DTC version lives on kitchen capacity, cold-chain logistics, and inventory spoilage. The marketplace version never touches a pan: it lives on having enough cooks near enough hungry diners, and on trust when a meal disappoints. Choosing the shape is choosing which operation you must become world-class at — and that choice is made before a single process is drawn.

Exercises

- 1.1. For each of the four company shapes, name one real business you use and state, in a sentence, the single operational question that most decides whether it survives.
- 1.2. Take one business idea of your own and describe how it would differ, operationally, if built as a marketplace versus as a DTC brand. Which operational capability does each version have to master?
- 1.3. A founder says “we’re a platform” but the company buys inventory, warehouses it, and ships it to consumers. Which shape is it **really**, and why does the mislabel put the operation at risk?
- 1.4. Classify each question as strategy, marketing, finance, or operations: (a) “should we enter Germany?”; (b) “what do we charge per box?”; (c) “who repacks an order when one item is out of stock?”; (d) “how do we get the launch trending?”

Answers. (1) A good answer pairs each shape with its life-or-death question: **marketplace** — is there enough liquidity that matches happen fast? **SaaS** — is it reliable and does a new user reach value before churning? **DTC** — can we source, hold, and ship physical units at margin without stocking out? **physical services / retail** — is there capacity in the right place at the right time? (2) The two versions must master **different** capabilities: a marketplace masters matching, trust, and dispute handling and touches no goods; a DTC brand masters supply, inventory, fulfilment, and

returns. (3) It is a **DTC** operation: buying, warehousing, and shipping physical units is DTC work, not marketplace matching. The “platform” label hides the real binding constraints (inventory, breakage, returns), so they go unmanaged. (4) (a) strategy; (b) finance / marketing; (c) operations; (d) marketing.

2. Systems thinking as an operational lens

2.1. A warehouse that optimised itself into failure

A fulfilment startup gave each of its three teams a clean, measurable target. **Receiving** was paid on pallets unloaded per hour. **Storage** was rated on how densely the shelves were packed. **Picking** was rated on orders shipped per shift. Every team beat its target. Every manager got a bonus. And the company missed its delivery promises so badly it nearly lost its biggest customer.

What happened? Receiving, racing to unload pallets, dumped goods at the dock without sorting them. Storage, maximising density, buried fast-moving items behind slow ones because that packed the cubes more tightly. Picking then walked miles per order and waited on mis-shelved stock. Each team **optimised its own number**; the **system** — the flow of a customer order from dock to doorstep — got slower. This is the single most important phenomenon in operations, and the rest of the course is, in a sense, a set of tools for not being this company.

2.2. Stocks and flows

The first move of systems thinking is to stop seeing an operation as an org chart and start seeing it as **stocks** connected by **flows**.

Definition 3 (stock and flow). A **stock** is anything that accumulates: inventory in a warehouse, orders waiting in a queue, cash in the bank, unresolved support tickets. It is a quantity measured **at an instant**. A **flow** is a rate that changes a stock over time: units arriving per hour, orders shipped per day, cash burned per month. A flow is measured **over an interval**.

The defining relationship — the one idea underneath all of inventory, queueing, and cash management — is simply that a stock is the running total of its flows. Measured over any period,

$$\text{stock at the end} = \text{stock at the start} + \text{total inflow} - \text{total outflow},$$

where the two totals are everything that arrived and everything that left during the period. In the plainest terms: **today's backlog equals yesterday's backlog, plus what arrived, minus what you cleared**. If inflow exceeds outflow, the stock grows — without limit, until something gives. This is why a support queue that receives 110 tickets a day and resolves 100 does not “run a little behind”; it accumulates an unbounded backlog, ten tickets deeper every single day.



Figure 1: The fundamental unit of systems thinking: a stock fed by an inflow and drained by an outflow. When inflow exceeds outflow the stock rises without bound. The bow-tie “valves” are the rates you can actually control.

2.3. Feedback, delay, and emergent behaviour

Stocks and flows rarely sit still, because flows depend on stocks. When the support backlog grows, customers escalate and call again — **raising** the inflow. That is a **feedback loop**: an output of the system loops back to become an input.

Definition 4 (feedback loop). A **feedback loop** is a path by which a change in a stock loops back, through some flow, to change that same stock again. A **balancing** (negative) loop counteracts change and pushes a stock toward a target; a **reinforcing** (positive) loop amplifies change, feeding growth on growth. Real systems are webs of both, and almost always contain **delays** between a change and the response it triggers.

A thermostat is the everyday balancing loop: too cold, so the heating comes on, which warms the room back toward the target. Hiring when the backlog grows is another. The support backlog above shows the reinforcing kind — backlog breeds re-contacts that breed more backlog. And the delays are everywhere: a new hire takes six weeks to become productive, so the correction arrives long after the problem was first observed.

When loops and delays combine, the system exhibits **emergent behaviour** — patterns that belong to the whole and cannot be read off any single part. Oscillation is the classic example: order too late, over-order to catch up, then sit on excess, then under-order... The famous **bullwhip effect**, where small swings in retail demand produce violent swings in factory orders upstream, is pure emergence from delay plus feedback. No single actor is irrational; the **structure** produces the swing.

Pitfall. The beginner’s instinct, when a system misbehaves, is to look for the bad actor or the broken part. Systems thinking insists you look first at **structure** — the loops, the delays, the stocks. Most chronic operational pain is structural, which is good news: structure is something you can **redesign**, whereas blame just relocates the symptom.

2.4. Suboptimization: the central warning

We can now name the warehouse’s disease precisely.

Definition 5 (suboptimization). **Suboptimization** is the degradation of the whole system’s performance caused by each part optimising its own local metric. It is not a failure of effort or competence — it is the predictable result of measuring and rewarding the parts as if the system were the sum of independent pieces. It is not.

The deep reason suboptimization is so common is that local metrics are **easy to measure and assign**, while system throughput is diffuse and shared. “Pallets unloaded” has an obvious owner; “did the customer get their order on time and cheaply” is everyone’s job and therefore no one’s. The systems-thinking corrective, which the Theory of Constraints in Chapter 3 makes operational, is to subordinate every local metric to a single **global** one — and to accept that some parts of the system **should** look “inefficient” so the whole can run fast.

Example — Reading a system before touching it. Before changing anything, sketch it as stocks and flows. For a meal-kit company: a stock of **raw ingredients**, a flow of **boxes assembled per hour**, a stock of **finished boxes awaiting pickup**, an outflow of **courier collections**. Immediately you can ask the right questions: which stock is growing? Which flow is the slowest? Where does a delay hide a problem until it is expensive? You have not yet computed anything, but you are already seeing the operation as a system — and you will never again be fooled by a team that is hitting its number while the company misses its promise.

Drawn out, that meal-kit company looks like Figure 2. Notice that it has **two** stocks, not one, chained by the flow between them — a real operation is almost always a chain of stocks like this, and mapping it is the first move of Chapter 2. When you draw your own, follow the same recipe: a rectangle for each thing that **accumulates**, a valved arrow for each **rate** that fills or drains it, and a cloud wherever the flow crosses the boundary of the part you are studying.

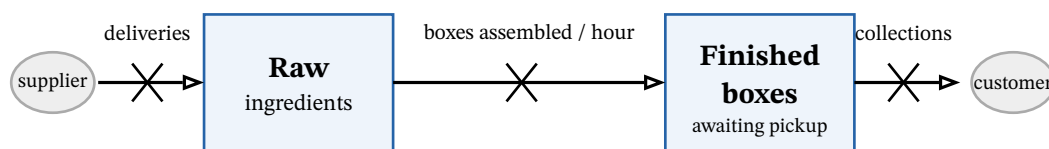


Figure 2: The meal-kit company as a stock-and-flow map. Two stocks — **raw ingredients** and **finished boxes awaiting pickup** — are chained by the **assembly** flow, filled by supplier **deliveries** and drained by courier **collections**. The map alone already poses the operational questions: which stock is growing, which flow is the slowest, and where does a delay hide a problem until it is expensive?

Exercises

2.1. Draw the stock-and-flow diagram for a coffee shop during the morning rush. Identify at least two stocks, two flows, and one feedback loop.

2.2. A SaaS support desk receives 90 tickets/day and closes 80/day, starting from a backlog of 50. Write the backlog as a function of day n . On what day does it reach 200? What does this tell you about “we’ll catch up next week”?

2.3. Give a concrete example, from any business you know, of a local metric that, when maximised, degrades the system. Name the global metric it should be subordinated to.

Answers. (1) Stocks: customers in line, orders taken but not yet served, cups awaiting pickup. Flows: arrivals/min, orders taken/min, drinks made/min, pickups/min. A balancing feedback loop: a long visible line makes some arrivals **balk** (leave), which lowers the inflow — the queue partly regulates itself. (2) Inflow exceeds outflow by 10/day, so $\text{backlog}(n) = 50 + 10n$. It reaches 200 when $10n = 150$, i.e. day $n = 15$. Because outflow is **below** inflow, there is no “catching up”: the backlog grows 10 every day without bound. “We’ll catch up next week” is false unless outflow is raised above inflow. (3) Any local metric works if you also name the global metric it should serve — e.g. maximising **server utilization** in a kitchen (local) at the cost of **order lead time** (global); the fix is to subordinate utilization to on-time delivery.

3. Constraints and the Theory of Constraints

3.1. One stage sets the pace

Picture a small cold-press juice line with four stages: **washing**, **pressing**, **pasteurising**, **bottling**. Their hourly capacities are 200, **80**, 150, and 130 litres. How much juice can leave the line per hour? Not 200, not the average — **80**. Pressing is the slowest stage, so it sets the pace for the entire line. Washing can sprint all it likes; it merely builds a pile of fruit in front of the press. Every system that transforms inputs into outputs has one stage like this, and recognising it changes everything.

Definition 6 (constraint / bottleneck). The **constraint** (or **bottleneck**) of a system is the single resource or policy that most limits the system's throughput toward its goal. By definition there is normally **one** binding constraint at a time: the stage whose capacity is lowest relative to demand. Everything else has, by comparison, **slack**.

This is the operational payoff of Chapter 2. Suboptimization happens because we improve non-constraints. Speeding up washing from 200 to 260 litres/hour costs money and produces **zero** extra juice, because pressing still caps the line at 80. An hour gained at the constraint, however, is an hour gained for the whole system.

Principle 1 (the leverage of the constraint). An improvement at a non-constraint yields no increase in system throughput. An improvement at the constraint flows through to the whole system, one-for-one, until the constraint moves elsewhere. Therefore the constraint is the highest-leverage point in the system, and the only place where local efficiency and global throughput coincide.

3.2. The Five Focusing Steps

The Theory of Constraints (TOC), due to Eliyahu Goldratt, turns that insight into a repeatable procedure. It is the backbone of operational diagnosis in this course.

Method 1 (The Five Focusing Steps).

1. **Identify** the system's constraint. Where does work pile up in front of, and starve behind? That stage is your bottleneck.
2. **Exploit** the constraint. Wring every drop from it **without spending money**: stop it idling at lunch, stop feeding it defective work, make sure it never waits.
3. **Subordinate** everything else to the constraint. Every other resource runs at the constraint's pace, not its own — even though that makes them look “underutilised”. This is the deliberate, healthy opposite of suboptimization.
4. **Elevate** the constraint. **Now** spend money: add a second press, hire, automate — raise the constraint's capacity.

5. **Repeat.** Once elevated, the constraint moves. Go back to step 1. **Do not let inertia become the new constraint** — the policy that “pressing is always our bottleneck” will blind you once it no longer is.

Pitfall. Steps 2 and 3 come **before** step 4 for a reason students routinely get wrong. The reflex is to **buy** capacity (elevate) the moment a bottleneck appears. But exploiting and subordinating are free and often recover 20–30% of constraint capacity that was being wasted on idling, rework, and mis-feeding. Spending money to elevate a constraint you have not yet exploited is paying to enlarge a leak.

3.3. Physical, policy, and organizational constraints

Identifying the constraint is harder than the juice line suggests, because not every constraint is a machine.

Definition 7 (constraint types). A **physical** constraint is a tangible capacity limit: a machine, a person, floor space, cash. A **policy** constraint is a rule, metric, or habit that limits throughput: “we batch all shipping to Fridays,” “every refund needs VP sign-off,” “utilisation must stay above 90%.” An **organizational** constraint is structural: handoffs between siloed teams, an approval chain, a misaligned incentive that no individual can override.

The crucial empirical fact: **most binding constraints in real businesses are policy or organizational, not physical.** The juice line’s true bottleneck might not be the press but the policy of pressing only one recipe per day (losing an hour to cleaning between switches), or the org structure that makes sales promise delivery dates production never agreed to. Policy constraints are simultaneously the most common and the cheapest to fix — you change a rule, not a machine — which is exactly why hunting for them is so valuable.

Example — Finding a policy constraint. An online lender complained its **physical** constraint was underwriter capacity: loans piled up awaiting human review. Before hiring (elevating), they ran the Five Steps. **Identify:** yes, work pools before underwriting. **Exploit:** underwriters spent 40% of their time chasing missing documents — a self-inflicted starvation. The real constraint was a **policy:** the application form did not require the documents up front. Fixing the form recovered nearly half the apparent shortage **for free.** Had they jumped to “hire more underwriters,” they would have paid to scale a broken process.

Exercises

- 3.1. A claims-processing line has stages with capacities 50, 30, 45, 60 files/day. Which is the constraint? If you could add 10 files/day of capacity to exactly one stage, which gives the largest throughput gain, and what is it?
- 3.2. For the same line, give one **exploit** action and one **subordinate** action that cost no money.
- 3.3. Classify each as physical, policy, or organizational: (a) a single MRI scanner; (b) a rule that no order ships without manager approval; (c) sales and operations using different demand forecasts. For each, sketch the cheapest plausible fix.

Answers. (1) System throughput is the smallest stage capacity, so the constraint is the **30 files/day** stage. Add the 10 there: it rises to 40, and the new system throughput is $\min(50, 40, 45, 60) = 40$ — a gain of **+10 files/day**. Adding the 10 to any other stage leaves the 30-stage binding, so throughput stays 30 and the gain is **zero**. Improve the constraint, nothing else. (2) **Exploit** (free): make sure the 30-stage is never starved or idle — stagger breaks so it always has work, and stop sending it files that will be rejected downstream. **Subordinate** (free): pace stages 1, 3, and 4 to 30/day so they neither pile WIP in front of the constraint nor starve it. (3) (a) **physical** — elevate by adding hours/a second scanner, or exploit by triaging urgent scans first; (b) **policy** — raise the approval threshold or delegate sign-off, a free rule change; (c) **organizational** — adopt a single shared forecast (one S&OP process) so the two functions plan against the same numbers.

4. Throughput, utilization, and the cost of being busy

4.1. Why the 95%-busy team is the slow one

Two support teams handle identical work. Team A runs at **70%** utilization; team B, praised by management for efficiency, runs at **95%**. Tickets resolve in two days at Team A and in **more than two weeks** at Team B. The “lazy” team is faster — dramatically. This is not an anomaly to explain away; it is the most important quantitative law in operations, and it is the reason “keep everyone busy” is one of the most expensive instincts in management. By the end of this chapter you will be able to say, with a number, exactly why.

4.2. The three core measures

Definition 8 (throughput, capacity, utilization). **Throughput** (X) is the rate at which the system actually produces finished output (orders/day, litres/hour). **Capacity** is the maximum possible throughput. **Utilization** (ρ) is the fraction of capacity in use:

$$\rho = \frac{\text{throughput}}{\text{capacity}}.$$

For the simplest case — a **single** server processing a queue in steady state — utilization is the same as the arrival rate divided by the service rate, $\rho = \lambda/\mu$, because capacity is then just one server’s service rate. With several servers, capacity is their combined rate, and ρ is throughput over that combined capacity. This single-server form is the one we return to when a worked example needs a number.

Definition 9 (cycle time). **Cycle time** (or **lead time**) is the total time one unit spends in the system, from entry to exit — including all the time it spends **waiting**, which is usually most of it. It is what the customer actually experiences as “how long it took.”

A trap worth dispelling immediately: throughput is **not** the same as utilization. A system can be 100% utilized and have terrible throughput toward the goal if it is busy producing the wrong thing, building inventory no one ordered, or reworking defects. Busy is not the same as productive — utilization measures **busyness**, throughput measures **goal-attainment**. TOC’s whole point is to subordinate the first to the second.

4.3. The utilization–delay curve

Here is why Team B drowns — and the culprit is not simply that it is busier. The real cause is **variability**. Work never arrives in a perfectly even trickle, and no two jobs take exactly the same time: arrivals clump, and service times differ. Whenever two jobs happen to land close together, the second must wait behind the first — and that momentary pile-up is the seed of every queue.

To see that variability, not utilization, is what **creates** the wait, imagine removing it. Suppose customers arrived perfectly regularly — one exactly every five minutes — and each took exactly four minutes to serve. Then every arrival finds the server free, waits **zero**, and this stays true even at 99% utilization: the system runs flat out and perfectly calm. With no variability there is no queue, at any utilization below 100%. Now let arrivals and service times vary even slightly and a queue appears at once — and the **same** variability that causes only a brief, self-correcting wait at 50% utilization causes an unbounded one as utilization climbs toward 100%.

So the law has two parts, and their order is the whole point: **variability creates the wait; utilization only sets how violently it is amplified**. At low utilization the server has idle gaps to absorb a clump and recover before the next one arrives; at high utilization it never catches up, so the delay from each clump piles onto the last. For a simple system with natural variability, the average time a unit spends **waiting in the queue** — call it W_q , to keep it distinct from the total time in the system — scales like

$$W_q \propto \frac{\rho}{1 - \rho}.$$

Look at what the $1 - \rho$ in the denominator does. At $\rho = 0.5$, the factor is $\frac{0.5}{0.5} = 1$. At $\rho = 0.9$ it is $\frac{0.9}{0.1} = 9$. At $\rho = 0.98$ it is 49. Going from 90% to 98% busy — a mere eight points of “efficiency” — **multiplies waiting by more than five**. The curve has a knee, and past it lies a cliff. But notice what the factor does **not** contain: were the variability zero, the whole wait would be zero however close ρ crept to 1. Utilization sits on the amplifier, never on the source.

This also settles the Team A versus Team B puzzle. Both teams handle the **same** variable work; what differs is only how much idle slack each keeps to absorb it. The ratio of their waiting factors is

$$\frac{0.95/0.05}{0.70/0.30} = \frac{19}{2.33} \approx 8.$$

So if Team A clears a ticket in about two days, Team B — eight times slower to wait — takes it past two weeks. The extra 25 points of utilization — running at 95% instead of 70% — cost a factor of eight in delay.

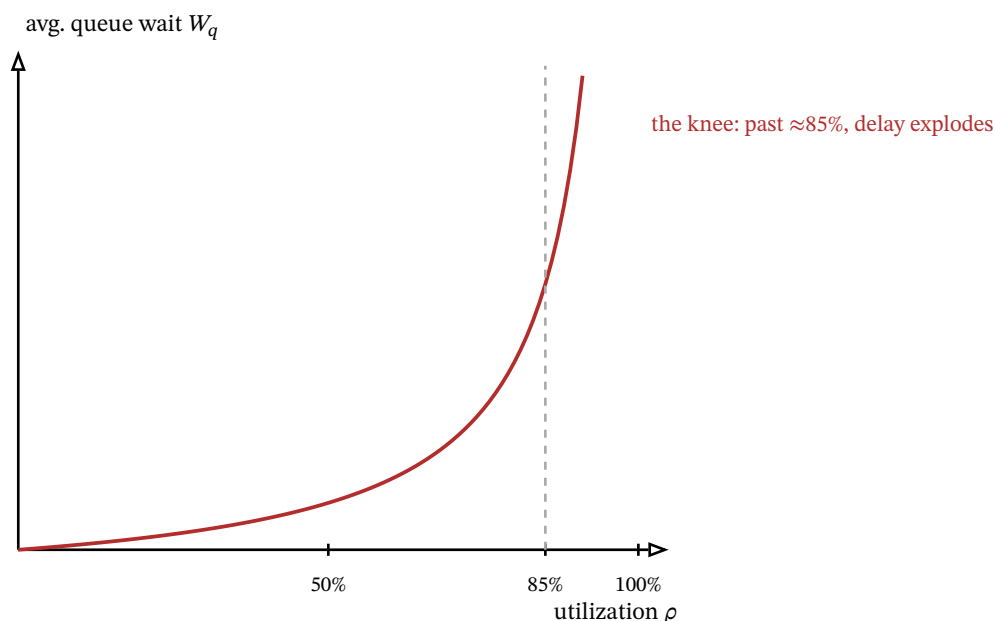


Figure 3: The utilization–delay curve, $W_q \propto \frac{\rho}{1-\rho}$, for a system with variability. Queue wait is mild and nearly flat until roughly 85% utilization, then rises vertically toward the wall at 100%. A system with **no** variability would sit flat on the axis at zero the whole way — the curve is the price of variability, and utilization only sets where on it you sit.

Principle 2 (variability makes the queue; the 85% rule of thumb). **Variability is what creates waiting.** A perfectly regular system — identical gaps between arrivals, identical service times — never forms a queue, even at 99% utilization. Add variability and a queue appears; utilization then decides only how far it grows. In any system with variability in arrivals or service times, waiting time (and therefore cycle time) grows without bound as $\rho \rightarrow 1$ — gently below roughly 85% utilization, catastrophically above it. Therefore a system you want to stay **responsive** must be run with deliberate spare capacity; “fully utilized” and “responsive” are mutually exclusive whenever the work varies at all.

Pitfall. The 85% figure is a **rule of thumb, not a law of physics**. The more variable the work — spiky arrivals, wildly differing job sizes — the **lower** the knee, sometimes 70% or less. The smoother and more predictable the work, the higher you can safely push. The durable lesson is the **shape** of the curve, not the exact number: delay is convex in utilization, so the last few points of “efficiency” are bought with disproportionate pain.

4.4. Slack as strategic capacity

This reframes idle capacity entirely. The 30% of “unused” time at Team A is not waste; it is **slack** — the buffer that absorbs variability and keeps cycle time short. Slack is what lets the system respond to a surge, recover from a disruption, and improve itself instead of forever firefighting. Eliminating slack to look efficient saves a little money now and charges a large bill later — and that bill falls due exactly when demand spikes and you can least afford it.

Example — Pricing the last 10%. A logistics manager proposes pushing sorting-hub utilization from 82% to 92% to “defer buying a second line.” Using $W_q \propto \frac{\rho}{1-\rho}$: at 82%, the factor is 4.6; at 92%, it is 11.5 — average dwell time **more than doubles**. Packages that cleared the hub in 6 hours now take 15. The saved capital is real; the cost is a service-level collapse that shows up as missed delivery promises and, soon, lost customers. Naming the trade-off quantitatively is the operations designer’s job; “just run it hotter” is not a plan.

Exercises

- 4.1.** A call centre’s agents are 96% utilized and customers wait 8 minutes on average. Using $W_q \propto \frac{\rho}{1-\rho}$, estimate the average wait if utilization were brought down to 80%. (Hint: take the ratio of the two $\frac{\rho}{1-\rho}$ factors.)
- 4.2.** Explain to a CFO, in two sentences, why running the warehouse at 99% utilization during peak season is a false economy. Use the shape of the curve, not jargon.
- 4.3.** A process is 100% utilized but throughput toward the goal is falling. Give two distinct operational explanations consistent with both facts.

Answers. (1) The factor at 96% is $0.96/0.04 = 24$; at 80% it is $0.80/0.20 = 4$. The wait scales with the factor, so the new wait is about $8 \times (4/24) = 8/6 \approx 1.3$ minutes — a sixfold drop for sixteen points of freed capacity. (2) A good answer stays on the curve’s shape: near 100% the wait curve is almost vertical, so each extra package added at peak adds hugely disproportionate delay; the “saved” capital of not adding a line is dwarfed by the missed-delivery costs when dwell time explodes. (3) Two distinct causes: (a) the system is busy on the **wrong** output — building unsold inventory or low-priority work, so utilization is high but goal-throughput is not; (b) a rising **rework/defect** rate is consuming capacity on redoing work, so the machine is 100% busy but fewer good units come out.

5. Queues, flow, and Little's Law

5.1. Counting a queue without a stopwatch

You walk into a clinic. Forty patients are in the waiting room, and you can see the desk admits about ten patients per hour. Roughly how long will you wait? You do not need to time anyone: $40 \div 10 = 4$ hours. You have just used the most useful equation in flow analysis, and it required nothing but a stock and a flow you could observe by standing still.

5.2. Little's Law

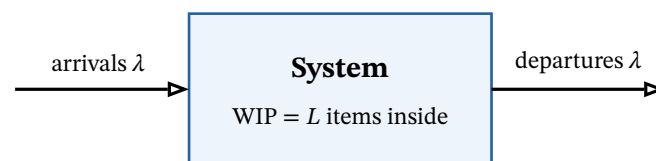
Theorem 1 (Little's Law). For any stable system — one whose backlog is not trending up or down over the period of interest — the average work-in-progress equals the average throughput times the average time in system:

$$L = \lambda \times W,$$

where L is the average number of items **in** the system (WIP), λ is the average **throughput** rate (arrivals = departures, since the system is stable), and W is the average **time** each item spends in the system.

Its power is its generality: it assumes nothing about the order of service, the distribution of arrivals, or the number of stages. It holds for patients in a clinic, orders in a factory, tickets in a queue, cars on a motorway, and cash tied up as receivables. Rearranged, it answers the three questions a designer actually asks:

$$W = \frac{L}{\lambda} \quad (\text{how long?}) \quad L = \lambda W \quad (\text{how much WIP?}) \quad \lambda = \frac{L}{W} \quad (\text{what rate?})$$



each item spends time $W = L/\lambda$ inside

Figure 4: Little's Law. In a stable system the inflow and outflow rates are equal (call it λ); the number of items inside (L) and the time each spends inside (W) are then locked together by $L = \lambda W$. Fix any two and the third is determined.

Pitfall. Little's Law requires **stability**: averages taken over a period where the queue is not systematically growing or shrinking. Apply it to a queue that is exploding (inflow > outflow, Chapter 2) and the “average WIP” is meaningless — there is no average, the stock is running away. Always check the queue is in steady state before you trust the number.

5.3. One process, all three numbers

Utilization, cycle time, and Little's Law describe the **same** process from three angles. It is worth carrying one process through all of them once, end to end.

Example — The passport-photo desk. A single clerk photographs and prints passport photos. Customers arrive at $\lambda = 12$ per hour; each takes on average 4 minutes, so the clerk's service rate is $\mu = 15$ per hour.

- **Throughput.** The queue is stable ($\lambda < \mu$), so departures equal arrivals: throughput = 12 customers/hour.
- **Utilization.** $\rho = \lambda/\mu = 12/15 = 0.8$ — the clerk is busy 80% of the time.
- **Cycle time.** For the simplest random single-server queue — one clerk, with arrivals and service times that are individually unpredictable but steady on average — the average number in the system works out to $L = \rho/(1 - \rho) = 0.8/0.2 = 4$ customers. This is exactly the $\rho/(1 - \rho)$ shape of Chapter 4, now with the constant pinned down. By Little's Law the time in system is $W = L/\lambda = 4/12$ h, that is **20 minutes**, of which 4 are the photo itself and **16 are spent waiting**. Those 16 minutes are the queue wait W_q of Chapter 4; adding the 4-minute service gives the total time in system W . Two different letters for two different quantities, related by $W = W_q + \text{service time}$.

Push the clerk to $\rho = 0.9$ (arrivals rise to 13.5/hour) and L jumps to $0.9/0.1 = 9$, so $W = 9/13.5$ h = 40 minutes — the wait doubles for a ten-point rise in utilization. Same clerk, same 4-minute photo; the queue, not the work, is where the time lives.

This idealised constant assumes one particular, **middling** kind of randomness — the everyday, unpredictable arrivals and service times of a typical single queue. A real system can be smoother than that and wait **less**, or burstier and wait **more**: the constant is a mid-range baseline, not a floor. (Recall Chapter 4: a **perfectly** regular system waits zero.) Use it to feel the **shape**, not to promise a customer 20 minutes to the second.

5.4. Visible and invisible queues

Physical queues — people in a room, boxes on a floor — are **visible**: you feel the pressure to act. The dangerous queues in modern operations are **invisible**: a backlog of 3,000 unprocessed emails, 800 open browser tabs of pending decisions, a six-week software backlog living in a database. They obey Little's Law exactly the same way — 3,000 emails cleared at 200/day is a 15-day wait — but because no one **sees** a pile, no one feels urgency, and the queue grows unmanaged until it becomes a crisis.

Remark. A central discipline of good operational design is to **make invisible queues visible**: a physical or digital board showing WIP, age of the oldest item, and inflow-vs-outflow. You cannot manage a stock you cannot see, and Little's Law only helps if you know L .

5.5. Batch size: the hidden delay multiplier

Why do invoices that “take five minutes to process” sit for three weeks? Because they are **batched**: held until month-end, then run together. Batching feels efficient — one setup, one run — but it injects delay directly into cycle time.

Example — The cost of the batch. A team processes expense reports only on the last Friday of each month. A report filed the day **after** a run waits about 26 days before anything happens — the **worst** case. Averaged over all filing dates, a report waits about **half** the cycle, roughly 13 days. Switch to processing daily — smaller batches — and that average wait collapses to under one day. Nothing about the **work** changed; only the **batch size**. Large batches also amplify the bullwhip effect from Chapter 2: lumpy batch releases look like demand spikes to whoever is downstream, who then over-react.

Principle 3 (small batches reduce delay). Reducing batch size reduces average cycle time and smooths flow, at the cost of more frequent setups. Where setup (or transaction) cost is low — most digital work — the delay savings dominate, and the right default is the smallest batch you can afford: ship one piece at a time. This is the operational core of “continuous flow” and of agile delivery.

5.6. Rework: the hidden capacity tax

Batching lengthens the wait without touching the work. A **rework loop** is nastier: it quietly destroys capacity that no one budgeted for. Whenever a step’s output is checked and some fraction is sent back to be redone, that step is secretly doing more work than its headline capacity suggests.

Definition 10 (rework loop). A **rework loop** returns a fraction of a step’s output to be processed again — a failed inspection, a bounced approval, a bug sent back to a developer. If a fraction r of each unit’s output must pass through the step a second time, then for every good unit the step performs $1 + r$ units of work, so its **effective** good-output capacity falls to roughly $C/(1 + r)$, where C is the raw capacity.

Example — A fifth of the work, looping back. A quality-check station can inspect $C = 120$ units/hour. But 20% of what it passes is rejected downstream and comes back for one more pass, so $r = 0.20$. Its effective capacity for **good** units is

$$120/(1 + 0.20) = 100 \text{ units/hour.}$$

A fifth of the work looping back once turned a 120/hour station into a 100/hour one — a sixth of capacity gone, invisibly, with no one idle. Worse, the extra circulating units **raise** WIP, so by Little’s Law cycle time climbs too. And if reworked units can fail **again** at the same rate, the loss compounds toward $C \times (1 - r)$. Rework is thus a double penalty: less throughput and longer delay, from a defect rate a dashboard rarely shows.

This connects back to feedback and delay from Chapter 2. Long queues **are** long delays, and delays destabilise feedback loops — the correction always arrives too late. Short queues mean fast feedback, which means the system can see and fix its own problems before they compound. Flow is not just about speed; it is about a system’s ability to learn.

Exercises

- 5.1.** A kanban board shows 24 cards in “in progress” and the team finishes 6 cards/day. What is the average time a card spends in progress? If the team wants that down to 2 days without working faster, what must L become, and how do you achieve it?
- 5.2.** Receivables average €600k outstanding; the firm collects €40k/day. What is the average collection period (days sales outstanding)? Interpret it via Little’s Law.
- 5.3.** A quality step handles 90 units/hour of raw capacity, but 25% of its output is returned for one more pass. What is its effective good-output capacity?
- 5.4.** A report is generated weekly from data that arrives continuously. Estimate the average “data age” in the report and explain how daily generation changes it.

Answers. (1) $W = L/\lambda = 24/6 = 4$ days. To reach $W = 2$ days with the same throughput ($\lambda = 6/\text{day}$), Little’s Law needs $L = \lambda W = 6 \times 2 = 12$: cut work-in-progress in half by limiting cards in progress to 12 — flow faster by starting less, not by working harder. (2) Days sales outstanding $= L/\lambda = 600/40 = 15$ days: cash behaves like WIP, so €600k of receivables “flowing out” at €40k/day spends 15 days in the system before it is collected. (3) With $r = 0.25$, effective capacity $= 90/(1 + 0.25) = 72$ good units/hour — an 18-unit loss no headcount would reveal. (4) Data collected uniformly over a week is on average **half a week** (≈ 3.5 days) old when the weekly report runs; generating daily cuts the average age to about half a day.

6. Designing a process from scratch

6.1. From a promise to a flow

Everything so far has been **analysis** — a way of seeing. Now we **design**. A business model makes a promise (“fresh meal kits delivered next-day”); a process is the concrete chain of steps that keeps that promise, every time, at volume. Designing one from scratch means drawing the flow that carries a single customer request from “I want this” to “I have it,” and then asking the course’s two questions of every link: **can it be delivered, and where does it break first?**

6.2. Mapping the value delivery flow

Start by mapping the path of **one unit of customer value** — one order — across the whole system, ignoring the org chart. The org chart tells you who reports to whom; the **flow** tells you how value actually moves, and value rarely respects departmental boundaries.

Definition 11 (process map). A **process map** is a directed diagram of the steps a unit passes through from request to delivery. Boxes are **activities** (they add value or transform the unit); arrows are **flow**; diamonds are **decision points** where the path branches on a condition; and the points where the unit passes from one owner to another are **handoffs**. A good map also marks where units **wait** — the queues between steps, which Chapter 4 and Chapter 5 told us are where the time actually goes.

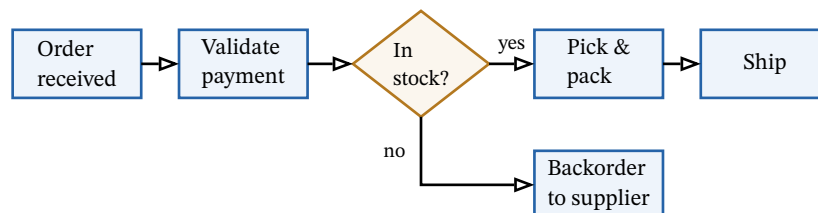


Figure 5: A process map for order fulfilment. Boxes are activities, the diamond is a decision point that branches the flow, and each arrow that crosses an ownership boundary is a handoff — the spots where work waits and information is lost.

6.3. Decision points and handoffs: where designs leak

Two features of a map deserve a designer’s paranoia.

Decision points are where the flow branches. Each one is a place to ask: who decides, on what information, how fast, and what happens to the “no” branch? An unstaffed or ambiguous decision point is where units silently pool into an invisible queue.

Handoffs are where a unit passes between people, teams, or systems. Every handoff is a risk: information is dropped, responsibility blurs, and a queue forms at the boundary while the receiver finishes something else. The empirical rule is blunt: **most delay and most defects are born at handoffs, not inside activities**. Therefore a core design move is to **minimise handoffs** — fewer boundaries crossed, less leakage.

Pitfall. A seductive error is to design the **happy path** only — the clean sequence when everything is in stock, payment clears, and the customer behaves. Real volume is made of exceptions. A process that has no explicit branch for “payment failed,” “out of stock,” or “customer changed the address after shipping” does not lack those cases; it just handles them by **improvisation**, which does not scale and is exactly where the design breaks first.

6.4. Designing for clarity

The aim is not a baroque diagram that captures every contingency; it is a process a tired human can execute correctly at 2 a.m. under load. **Designing for clarity** means: each step has one owner; each decision has an explicit rule; the default path is obvious; and the exceptions have named, owned routes rather than “someone will figure it out.” Clarity is itself a form of capacity — an ambiguous process burns time on coordination (the subject of Chapter 8) that a clear one spends on throughput.

Example — Designing, not documenting. A subscription-box startup mapped its fulfilment and found **eleven** handoffs between order and dispatch, three of them crossing into a separate “exceptions team” whose queue was invisible. Redesigning so that one fulfilment owner carried an order end-to-end — with explicit rules for the two genuinely hard exceptions — cut handoffs to four. Cycle time fell by half, with **no** new hardware and no faster work: the time had been living in the gaps between steps, exactly where Little’s Law said to look.

Exercises

- 6.1.** Map, as boxes/arrows/diamonds, the process of a customer returning a defective product for a refund. Mark every handoff and every decision point.
- 6.2.** For your map, identify the single handoff most likely to lose information, and propose a redesign that removes or de-risks it.
- 6.3.** Take a “happy path” process you know and add the three most likely exception branches. Which one, if unhandled, breaks the design at the lowest volume?

Answers. (1) A solid map runs: **request return** → decision **within return window?** (no → reject with reason) → **print label / ship back** (handoff: customer → carrier) → **receive & inspect** (handoff: carrier → warehouse) → decision **defective?** (no → restock or return to sender) → **issue refund** (handoff: warehouse → finance) → **notify customer**. Every arrow crossing an owner is a handoff; the two diamonds are the decision points. (2) Usually the **carrier** → **warehouse** handoff (the returned item arrives with no link to the original order) or **warehouse** → **finance** (inspection result doesn’t reach whoever refunds). De-risk by carrying a single return ID end-to-end so each owner passes it on with the item. (3) Depends on the process, but the branch that breaks **first** is the exception that occurs **most frequently at low volume** — commonly “payment failed” or “item out of stock,” because those happen from day one, whereas rarer exceptions only bite at higher volume.

7. Service design

7.1. The operation the customer actually feels

A process map (Chapter 6) shows how work flows. But when the “product” is a **service** — a haircut, a clinic visit, an onboarding flow, a food delivery — the customer is **inside the operation while it runs**. They experience the queue, the handoffs, the failures, in real time. Service design is process design seen from the customer’s seat: it asks not only “does the work get done?” but “what is it like to be the unit moving through this system?”

7.2. Touchpoints and the line of visibility

Definition 12 (touchpoint, frontstage, backstage). A **touchpoint** is any moment the customer interacts with the service — a webpage, a confirmation text, a driver at the door, a support chat. **Frontstage** actions are the ones the customer sees; **backstage** actions are the work that enables them but stays hidden, separated by a conceptual **line of visibility**. A **service blueprint** maps customer actions, frontstage, and backstage together, so each visible touchpoint is traced to the hidden work that must succeed for it to land.

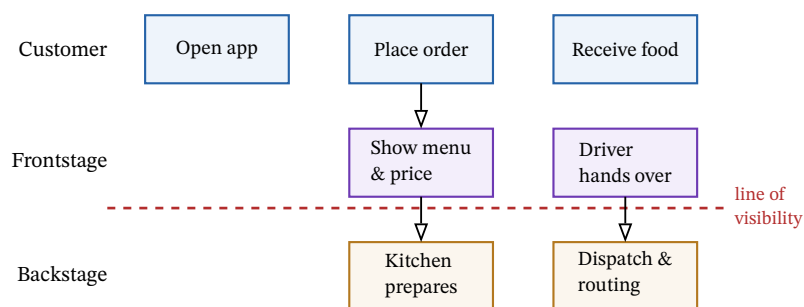


Figure 6: A service blueprint for food delivery. Above the dashed line of visibility are the customer’s actions and the frontstage touchpoints they see; below it is the backstage work that must succeed for each touchpoint to land. Every frontstage promise has a backstage dependency — and every dependency is a potential failure point.

7.3. Failure points and recovery paths

The reason to draw the blueprint is to find where it can fail **before the customer does**.

Definition 13 (failure point and recovery path). A **failure point** is a touchpoint or backstage step that can go wrong from the customer’s perspective: the driver is late, the dish is wrong, the payment is declined, the page errors. A **recovery path** (service recovery) is the designed response that returns a failed interaction to an acceptable outcome — a proactive apology and refund, a fast re-send, a human escalation.

A foundational result of service research, the **service-recovery paradox**, is that a failure handled excellently can leave a customer **more** loyal than one who never hit a problem at all — whereas a

failure handled badly loses them for good. The implication for design is sharp: **recovery is not an afterthought, it is part of the service**. A blueprint without recovery paths is a design that works only when nothing goes wrong, which at volume is never.

Pitfall. Designers lavish attention on the **happy-path** touchpoints — the slick onboarding, the beautiful confirmation screen — and leave failure handling to improvised customer-service heroics. But customers judge a service disproportionately by **how it behaves when it breaks**. An un-designed recovery path is the most common reason a technically-functional operation still bleeds customers.

Example — Designing the failure, not just the feature. A telehealth startup blueprinted a consultation and listed failure points: patient can't find the video link, doctor runs late, connection drops mid-call, prescription doesn't reach the pharmacy. For each they designed a recovery: an SMS link fallback, a live "your doctor is 6 min away" status, a one-tap phone-call fallback, and automated pharmacy confirmation with a human chaser. None of these are the "product" — and they are exactly what made patients trust it enough to return. The competition designed the consultation; this team designed the consultation **and its failure modes**.

Exercises

7.1. Blueprint a hotel check-in: list at least four customer actions, the frontstage touchpoints, and one backstage step behind each. Draw the line of visibility.

7.2. For that blueprint, identify the three most likely failure points and design a recovery path for each.

7.3. Explain the service-recovery paradox to a founder who wants to cut the support budget because "the product just works."

Answers. (1) Customer actions: arrive, queue at desk, give name/ID, receive key, go to room. Frontstage: receptionist greeting, booking lookup on screen, key handover, directions. One backstage step behind each: booking synced from the reservation system; room marked clean by housekeeping; key card encoded; room actually vacated and serviced. The line of visibility sits between the receptionist's screen (frontstage) and the housekeeping and PMS systems (backstage). (2) Likely failure points and recoveries: **reservation not found** → search by email/card and honour the rate on the spot; **room not ready** → proactive apology, store luggage, offer a drink, text when ready; **key card fails** → staff escort and re-encode immediately. (3) A good explanation: customers who never hit a problem feel neutral; a customer whose problem is fixed fast and graciously often ends up **more** loyal than one who sailed through — so support is not a cost to cut but the part of the service that earns the most loyalty per euro, precisely when things break.

8. Make vs. buy

8.1. Build it or rent it?

Your meal-kit company needs a delivery capability. Do you hire drivers and buy vans (**make**), or contract a courier (**buy**)? The answer is not “whichever is cheaper per delivery,” because the visible price is the smallest part of the decision. Make-vs-buy is where unit economics, process design, and the constraint all collide, and getting it wrong either starves your real constraint of cash or hands a rival control of your customer experience.

8.2. The true cost of each option

Definition 14 (make vs. buy). **Make** means building and operating the capability internally. **Buy** means sourcing it from an outside provider. The decision compares the **total** cost and risk of each — not the sticker price — including the costs that do not appear on an invoice.

The reason naive make-vs-buy goes wrong is that each option hides a different cost:

Making hides **coordination cost**. Bringing a capability in-house adds headcount, management overhead, hiring and training, idle capacity when demand dips, and the attention of leadership — a scarce resource in a young company. These are real and largely fixed, and they are exactly the costs spreadsheets omit.

Buying hides **coordination cost of a different kind plus dependency risk**. You now manage a vendor relationship (contracts, SLAs, escalations — coordination across a company boundary, which is harder than across a team boundary), you inherit the vendor’s quality and failure modes, and you accept **strategic dependency**: if the capability is core to your promise, you have handed a supplier leverage over your business.

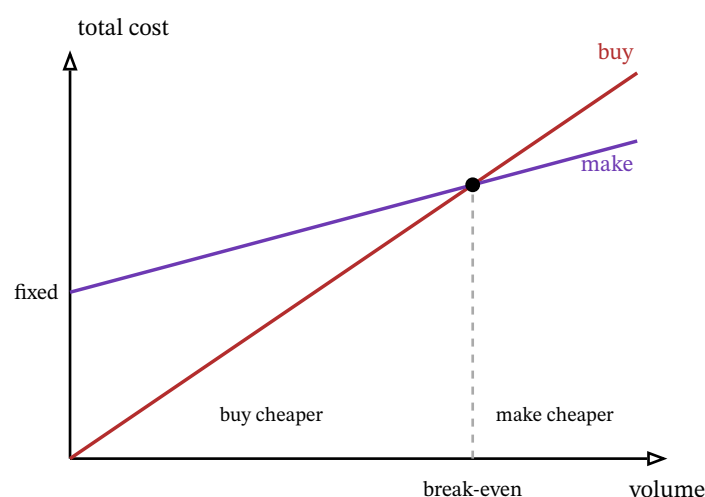


Figure 7: Make vs. buy as a cost crossover. **Buy** has no fixed cost but a high price per unit; **make** carries a fixed cost but a low variable cost. Below the break-even volume, buying is cheaper; above it, making is. The hidden costs — coordination on the make side, dependency and quality risk on the buy side — shift both lines, which is why the sticker-price crossover is never the whole decision.

Principle 4 (the make-vs-buy comparison). Choose **make** when total internal cost — including coordination and management overhead — is below total external cost **including** dependency and quality risk, **or** when the capability is strategically core and must not be outsourced at any reasonable price. Choose **buy** when an outside provider has genuine scale or specialisation you cannot match, the capability is non-core, and the dependency risk is bounded. The error to avoid is comparing internal **variable** cost against external **all-in** price — comparing two costs that do not cover the same things, which almost always flatters “make.”

8.3. Coordination cost and the core/non-core line

The deepest principle here is that **every boundary you create has a coordination cost**. Splitting work across an internal team boundary or an external vendor boundary both require communication, handoffs (Chapter 6), and reconciliation — and the cost rises with how tightly coupled and how variable the work is. This is why some things are cheaper to make even when a vendor’s unit price is lower: the coordination tax across the boundary exceeds the saving.

The second principle is the **core vs. non-core** distinction. Outsource what is **generic** (payroll, cloud hosting, ground shipping for most firms) to specialists who do it at scale. Keep **core** what **is your edge** — the part of the operation the customer is actually paying for and that differentiates you. A premium-experience brand outsourcing its customer interaction to the cheapest call centre is “saving money” by cutting away the very thing it sells.

Pitfall. “Buy, because it’s cheaper per unit **today**” ignores two things that bite later: the vendor will re-price once you depend on them, and a capability you never built is one you never learned. Outsourcing your **constraint** or your **core** trades a short-term cost saving for long-term loss of control over precisely where your business lives or dies.

Example — Buy now, make later. An early fintech outsourced its KYC identity checks to a SaaS vendor — clearly correct at launch: building it was months of work irrelevant to proving the business. Two years on, KYC was 30% of cost-per-customer, the vendor raised prices, and identity verification had become **core** to the firm’s risk edge. They brought it in-house. Both decisions were right **for their moment**. Make-vs-buy is not a one-time verdict; it is re-decided as volume, cost structure, and what counts as “core” all evolve.

Exercises

8.1. A startup can hire a 2-person support team for €8k/month (handling up to 1,000 tickets) or outsource at €6/ticket. At what monthly volume does make become cheaper on **direct** cost? Name two costs this break-even ignores on each side.

8.2. For a business you know, name one capability that should clearly be **bought** and one that should clearly be **made**. Justify each via core/non-core and dependency risk.

8.3. Explain how outsourcing the **constraint** of a system can reduce reported cost while increasing real operational risk.

Answers. (1) Set the direct costs equal: $8000 = 6V$, so $V \approx 1333$ tickets/month — below that, buying is cheaper; above it, making is. But note the in-house team caps at 1,000 tickets, and $1333 > 1000$, so within one team’s capacity buying is always cheaper on direct cost (€6,000 vs €8,000 at 1,000); making only wins once you add a second team.

Ignored costs — **make**: coordination/management overhead, hiring and training, idle capacity at low volume; **buy**: vendor management and SLAs, quality risk, dependency. (2) A good answer **buys** something generic and non-core with a bounded, replaceable vendor (payroll, cloud hosting) and **makes** something core and differentiating whose dependency risk would be unacceptable (the recipe, the recommendation engine, the core customer interaction). (3) Outsourcing the constraint can lower the invoice per unit, but the constraint sets the **whole system's** throughput; handing it to a vendor means your growth ceiling, quality, and pricing at the one place that governs the system are now controlled by someone else — cheaper on paper, far riskier in reality.

9. Unit economics as operational constraints

9.1. The margin that bounds the design

A grocery-delivery startup wants a delightful operation: hand-checked produce, a phone call for every substitution, same-hour delivery. Wonderful — and impossible, because a €40 basket leaves only about €6 of contribution once the groceries, standard picking, delivery, payment fees, and basic support are paid for — and every one of those delightful touches has to come out of that same €6. Unit economics are not the finance team's concern to be handled “later”; they are a **hard constraint on what the operation is allowed to be**. Every process choice in the previous chapters spends money per unit, and the margin sets the budget.

9.2. The unit-economics toolkit

Definition 15 (unit economics). **Unit economics** describe the profit of a single unit of business (an order, a customer, a ride). The core quantities:

- **Contribution margin** per unit = price – variable cost — what each sale contributes toward fixed costs and profit.
- **Fixed costs** — costs that do not vary with volume (rent, salaried staff, software).
- **Variable costs** — costs that scale with each unit (materials, payment fees, per-order labour and delivery).
- **Break-even volume** = $\frac{\text{fixed costs}}{\text{contribution margin per unit}}$ — the number of units at which total contribution covers fixed costs.
- **CAC** (customer-acquisition cost) — what it costs, on average, to win one customer. **LTV** (lifetime value) — the total contribution a customer delivers before they leave, i.e. the baseline contribution margin summed over every purchase they make. In this course both arrive as **given inputs** — numbers handed to you rather than derived here.

Read operationally, LTV – CAC is the lifetime **profit** left per customer once acquisition is paid for — the headroom that any **extra** per-customer investment, over and above the baseline operation already priced into the contribution margin, must come out of; a healthy venture keeps LTV/CAC comfortably above one.

The contribution margin is the **per-unit budget** every operational decision draws against. A “small” design choice — a phone call per order, a premium courier, a second quality check — that costs €3 per unit is not small if the contribution margin is €6: it has spent half the budget. This is the discipline the course demands: translate every operational ambition into its per-unit cost and check it against the margin **before** committing to it. The same logic runs over a customer's lifetime, with one caution: the baseline cost of serving a customer is already inside the contribution that builds their LTV, so what remains is profit, not a fresh serving budget. If acquiring a customer costs €30 (CAC) and their lifetime value is €120, then €90 of lifetime profit is left — the headroom for any **extra** investment in that customer, such as a concierge call, a loyalty perk, or proactive outreach, before the relationship turns unprofitable.

9.3. Fixed vs. variable: the shape of the cost structure

Whether a cost is fixed or variable changes the operational strategy, not just the arithmetic.

Principle 5 (cost structure bounds process design). A **fixed-cost-heavy** operation (owned warehouse, salaried staff, owned fleet) has low variable cost per unit but high break-even volume: it is punishing at low volume and extremely profitable once volume clears break-even, because each extra unit is nearly all contribution. A **variable-cost-heavy** operation (outsourced, pay-per-use) has a low break-even and survives easily at low volume, but its margin never improves with scale — there is no operating leverage. The right structure depends on how certain and how large your volume is, which ties directly back to the make-vs-buy decision of Chapter 8.

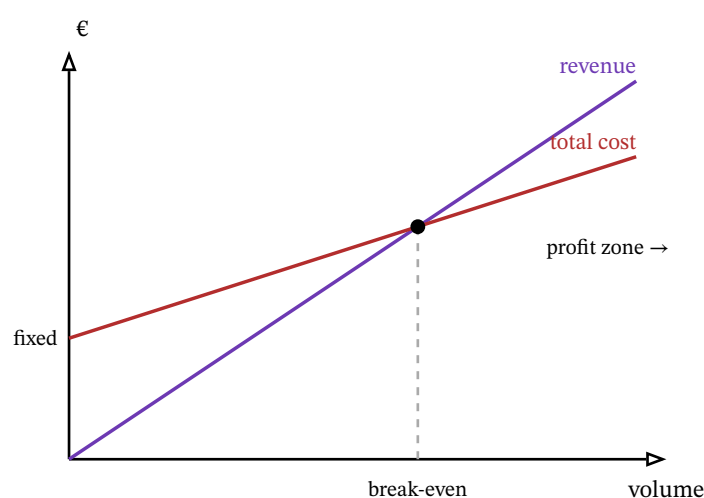


Figure 8: Break-even. Fixed costs lift the cost line to a positive intercept; the gap between the revenue slope (price) and the variable-cost slope is the contribution margin per unit. The lines cross at the break-even volume; beyond it, every unit's full contribution falls to profit. Higher fixed costs raise the intercept and push break-even to the right, but leave the profit zone's slope untouched — that slope is the contribution margin, set by price and variable cost alone. What widens the profit zone is a **lower variable cost** per unit, not a higher fixed cost.

Pitfall. A frequent founder error is to design the operation for the **margin they hope to have at scale** rather than the **margin they have now**. Slack capacity, premium service, and generous handling all consume contribution that an early-stage, low-volume operation does not yet have. Design to the **current** unit economics; earn the richer operation as volume and margin actually arrive.

Example — Unit economics killing a feature. A same-day flower service wanted live SMS updates with a human on standby for re-routes. Costed out: about €2.10 per order in staff time. Contribution margin per order: €4.50. The feature would have consumed **47%** of the budget that also had to cover delivery failures, refunds, and overhead. They shipped automated SMS (€0.05) and reserved the human for the $\approx 5\%$ of orders that genuinely needed it. Same customer outcome on the cases that mattered, at a fortieth of the cost. The unit economics did not **inform** the design — they **bounded** it.

Exercises

- 9.1.** A meal kit sells for €35 with €24 of variable cost; fixed costs are €22k/month. What is the contribution margin and the monthly break-even volume? How many kits must it sell to make €10k profit?
- 9.2.** The founder wants to add a hand-written note (€0.80) and premium packaging (€1.50) per kit. By how much does break-even volume rise? Is it worth it — what would you need to know?
- 9.3.** A customer costs €30 to acquire (CAC) with a lifetime value of €120. The team proposes a concierge onboarding call costing €25 per customer. What does this do to the lifetime profit per customer, and what would justify it?
- 9.4.** Contrast a fixed-cost-heavy and a variable-cost-heavy version of the same delivery operation. At what kind of volume does each win, and why?

Answers. (1) Contribution margin = $35 - 24 = €11$. Break-even = $22000/11 = 2000$ kits/month. For €10k profit, cover fixed + target: $(22000 + 10000)/11 = 32000/11 \approx 2910$ kits. (2) The additions cost €2.30, cutting the margin to $11 - 2.30 = €8.70$. New break-even = $22000/8.70 \approx 2529$ kits — a rise of about **529 kits** ($\approx 26\%$). Worth it only if the note and packaging lift price, retention, or referrals enough to cover roughly 530 extra kits of break-even; you'd need the effect on repeat rate or word-of-mouth (i.e. on LTV). (3) Lifetime profit before the call is $LTV - CAC = 120 - 30 = €90$; the €25 call is an **extra** touch on top of the baseline operation, so it spends more than a quarter of that profit, leaving €65 of headroom for any other discretionary investment in the customer. It is justified only if the call raises LTV by clearly more than €25 — e.g. by cutting early churn enough to add more than €25 of lifetime contribution. (4) The **fixed-cost-heavy** version (owned fleet) wins at **high, certain** volume, where operating leverage makes each extra delivery nearly all contribution; the **variable-cost-heavy** version (outsourced couriers) wins at **low or uncertain** volume, where a low break-even survives the lean months — at the price of no margin improvement as it scales.

10. Operational MVP and failure-mode analysis

10.1. Shipping the smallest operation that proves the promise

You have analysed and designed. The last discipline is restraint: do **not** build the whole beautiful operation before you know it works. Just as a product gets a minimum viable **product**, an operation gets a minimum viable **operation** — the smallest version that lets you actually deliver to real customers and learn where it breaks, before you have sunk capital into automating something that should not exist.

10.2. The operational MVP

Definition 16 (operational MVP). An **operational MVP** is the minimum operational capability that can deliver the core promise to real customers and generate real learning — the smallest operation you genuinely run, not the whole intended operation in miniature.

Designing one means sorting every step of the intended operation into three buckets: **build** it now (it is essential and must be real), **simulate** it now (deliver the outcome manually or with a stand-in rather than the eventual automated system), or **defer** it (it is not yet needed at this volume). The key move is **simulation** — often called “doing things that don’t scale.” Before building a €200k automated warehouse system, fulfil the first hundred orders by hand from a spare room. The customer gets the same outcome; you spend almost nothing; and you learn what the real process needs from observing it, rather than guessing at a whiteboard. You build the expensive, automated version only for steps that simulation has **proven** are necessary and are about to become the constraint.

Principle 6 (the MVP scoping rule). **Build** only what is essential to the core promise and cannot be faked. **Simulate** everything whose outcome can be delivered manually at current volume, deferring its automation until volume makes the manual version the constraint. **Defer** everything not yet needed. The justification for each placement is a **volume**: “we will build X when we exceed Y orders/day, because that is when simulating it breaks.” Scoping without a volume trigger is just opinion.

Pitfall. The twin errors are **over-building** and **under-building**. Over-building automates and polishes operations for a scale you have not reached, burning the very cash (Chapter 9) your real constraint needs — the most common way well-funded operations starve. Under-building fakes a step that genuinely had to be real (safety, compliance, the core quality the customer is paying for) and ships a broken promise. Correct scoping is honest about **which** steps are which, and says so out loud.

10.3. Early failure-mode analysis

An operational MVP is not a hope; it is a hypothesis you **stress-test** on paper before volume tests it for you. This is the course's second question made systematic: **where will this break first, at what volume, and what do we do about it?**

Definition 17 (failure-mode analysis). A **failure-mode analysis** for an operational design enumerates the ways it can break, estimates the **volume or condition** that triggers each, ranks them by likelihood and impact, and proposes a **mitigation** for each. The disciplined output names the **first three failure points** — the ones that trigger soonest as the operation scales.

The reason to estimate a **triggering volume** for each failure, rather than just listing risks, is that it turns vague anxiety into a roadmap. “The packing table jams at ≈ 120 orders/day; the payment-reconciliation spreadsheet breaks at ≈ 500 customers; the single support agent saturates at ≈ 40 tickets/day” tells you **exactly** what to fix and **when** — in the order the failures will actually arrive. This is the Theory of Constraints applied forward in time: as you elevate one constraint, you are naming the next one before it bites.

Example — A failure-mode analysis that set the roadmap. A craft-soda startup ran the analysis on its hand-built operation. **Failure 1:** the single bottling line caps at ≈ 300 bottles/day — hits at ≈ 25 orders/day (week 3). Mitigation: a second hand-line, €400. **Failure 2:** manual address entry produces $\approx 2\%$ mis-deliveries — tolerable now, intolerable past ≈ 50 orders/day. Mitigation: address validation, deferred. **Failure 3:** the founder personally answers every support email — saturates at ≈ 30 /day. Mitigation: templates now, a hire at ≈ 60 orders/day. The analysis did not just list risks; it **sequenced the next three months of operational work** by triggering volume.

10.4. Second-order effects: fixing one thing reveals the next

A final, course-defining idea ties the whole sequence together. Operational improvements have **second-order effects**: solving the visible problem changes the system, often revealing or even worsening a problem elsewhere. This is not a reason to do nothing — it is a reason to **anticipate**.

Principle 7 (the moving constraint). Relieving a constraint does not “fix” the system; it **moves** the constraint. Throughput rises until the next-tightest resource binds — which is often hidden because it was never stressed before. Therefore every improvement should be paired with the question: **if this works, what becomes the new bottleneck, and are we ready for it?** Local efficiency gains, pursued without this question, can even degrade the whole — speeding a feeder floods the next stage's queue and, via the utilization–delay curve (Chapter 4), can lengthen overall cycle time.

Example — The improvement that made things worse. A warehouse doubled picking speed — its obvious bottleneck. Throughput barely moved, and **packing** now drowned: it had quietly been the second constraint, and a flood of picked orders pushed it past the knee of its utilization curve, so total cycle time **rose**. The fix to picking was correct in isolation and harmful in the system, precisely because no one asked where the constraint would move. **Always solve one constraint while watching for the next.**

10.5. Presenting an operational design to a non-operations reader

An operational design is worthless if it cannot be **defended** to the people who fund and depend on it — investors, co-founders, strategy leads — who do not speak in utilization curves and Little’s Law. Translating system dynamics into business consequences is the last skill of the course, and it has a reliable shape. Name four things, in the language of money and promises:

Method 2 (The four-part design justification).

1. **The constraint** — the one resource or policy that sets the pace, in plain terms.
2. **The triggering volume** — the specific level of demand at which it binds.
3. **The business consequence** — what the customer and the P&L feel when it does, quantified.
4. **The mitigation and its cost** — the concrete fix and what it takes to deploy it.

The difference this makes is the difference between a shrug and a decision. Compare “utilization will hit 95%” — true, and meaningless to an investor — with the same fact dressed for its audience:

Example — The same fact, made fundable. “At our forecast growth we run out of responsive capacity in March (the **constraint**: our single sorting line, binding at ≈ 900 orders/day — the **triggering volume**). When it binds, deliveries slip from 2 days to 2 weeks and we lose the enterprise customers who are 40% of revenue (the **business consequence**). The fix is a second sorting line at €18k, installed in six weeks — so we must commit by January (the **mitigation and its cost**).” Every clause is a system fact translated into a decision the reader can actually make. That translation — not the diagram — is what gets the operation funded.

This is the whole course in one motion: see the operation as a system, find where it breaks first, put a volume on it, and say what that costs the business — so that the two questions you started with can finally be answered out loud, in numbers, to the people who fund and depend on the answer.

Exercises

- 10.1.** For a new juice-bar delivery business, sort eight operational steps into build / simulate / defer, and give a volume trigger for each “build” and each “defer”.
- 10.2.** Run a failure-mode analysis on that MVP: name the first three failure points, the volume each triggers at, and a mitigation for each.
- 10.3.** Pick one mitigation that relieves a constraint and predict the second-order effect: what becomes the new constraint, and at what volume?
- 10.4.** Write a three-sentence briefing of your design for a non-operations investor, using the four-part shape: constraint, triggering volume, business consequence, and the fix.

Answers. (1) A good answer **builds** only the essential, un-fakeable steps (taking the order and payment, making and delivering the drink safely) and gives each a volume trigger for when the manual version must be automated; **simulates** what can be done by hand now (routing by phone, a spreadsheet “inventory”); and **defers** what current volume does not need (a custom app, a second kitchen) — each deferral tagged with the volume that will call for it. (2) The three failure points must each carry a **triggering volume** and a **mitigation**, ordered by which binds first — e.g. the blender saturates at ≈ 40 drinks/hour, the single driver at ≈ 15 deliveries/hour, the manual payment reconciliation at ≈ 200 orders/day. (3) A correct answer names a **new** constraint created by the fix and its volume — e.g. adding a second driver relieves delivery but pushes the blender past its knee at ≈ 40 drinks/hour, which becomes the binding constraint. (4) The briefing should hit all four parts in plain money-and-promises language, e.g.: “Our one blender caps us at 40 drinks/hour; we hit that at ≈ 300 orders/day in about two months. Past it, wait times blow out and we start refunding late orders — roughly €4k/month in lost contribution. A second blender at €900 buys us to ≈ 600 orders/day, so we should order it now.”

Primary sources and further reading

This course draws on a handful of primary sources; each is worth reading in the original.

- **Systems thinking.** Donella H. Meadows, *Thinking in Systems: A Primer* (2008) — stocks, flows, feedback, and suboptimization in plain language. Chapter 1 underpins Chapter 2.
- **The Theory of Constraints.** Eliyahu M. Goldratt & Jeff Cox, *The Goal* (1984) — the constraint idea told as a novel; and Eliyahu M. Goldratt, *Theory of Constraints* (1990), ch. 1, which states the Five Focusing Steps formally. Both stand behind Chapter 3. Note Goldratt uses “throughput” to mean **money per unit time**, narrower than the units-per-time sense used in Chapter 4 and Chapter 5.
- **Little’s Law.** John D. C. Little, “A Proof for the Queuing Formula: $L = \lambda W$,” *Operations Research* 9(3), 1961 — the original proof behind Chapter 5.
- **Process, capacity, and supply networks.** Nigel Slack, Alistair Brandon-Jones & Nicola Burgess, *Operations Management* (10th ed., 2022) — the general reference behind Chapter 4, Chapter 6, Chapter 8, and Chapter 9.
- **Service design.** Marc Stickdorn et al., *This Is Service Design Doing* (2018) for blueprints, touch-points, and recovery; and Frances X. Frei, “The Four Things a Service Business Must Get Right,” *Harvard Business Review*, 2008. Both inform Chapter 7.
- **The operational MVP.** Steve Blank, “Why the Lean Start-Up Changes Everything,” *Harvard Business Review*, 2013 — the source for “doing things that don’t scale” behind Chapter 10.